

```
--- djangoapi\scripts\crop\myLib\gestor.py ---
```

```
from myLib.connect import connect
from myLib import p1Settings
```

```
class tablasP00():
```

```
    def __init__(self):
        self.conn = connect()
        self.cur = self.conn.cursor()
        self.epsg = p1Settings.EPSG_CODE
```

```
    def disconnect(self):
        self.cur.close()
        self.conn.close()
```

```
    def geom_isValid(self, geom_wkt):
        query = "SELECT ST_IsValid(ST_GeomFromText(%s, %s));"
        self.cur.execute(query, [geom_wkt, self.epsg])
        return self.cur.fetchone()[0]
```

```
    def snap_to_grid(self, geom_wkt, size=p1Settings.TOLERANCE):
        query = "SELECT ST_AsText(ST_SnapToGrid(ST_GeomFromText(%s, %s), %s));"
        self.cur.execute(query, [geom_wkt, self.epsg, size])
        return self.cur.fetchone()[0]
```

```
    def snap_line_to_existing_network(self, table_name, geom_wkt, exclude_id=None,
                                      size=p1Settings.TOLERANCE):
```

```
        if exclude_id is None:
```

```
            query = f"""
```

```
                WITH input_geom AS (
                    SELECT ST_GeomFromText(%s, %s) AS geom
                ),
```

```
                network_geom AS (
                    SELECT ST_Collect(geom) AS geom
                    FROM {table_name}
                ),
```

```
                endpoint_adjusted AS (
```

```
                    SELECT
```

```
                        CASE
```

```
                            WHEN network_geom.geom IS NULL THEN input_geom.geom
```

```
                            ELSE ST_SetPoint(
```

```
                                ST_SetPoint(
```

```
                                    input_geom.geom,
```

```
                                    0,
```

```
                                    CASE
```

```
                                        WHEN ST_DWithin(ST_StartPoint(input_geom.geom),
```

```
network_geom.geom, %s)
```

```
                                        THEN ST_ClosestPoint(network_geom.geom,
```

```
ST_StartPoint(input_geom.geom))
```

```
                                        ELSE ST_StartPoint(input_geom.geom)
```

```
                                    END
```

```
                                ),
```

```

        ST_NPoints(input_geom.geom) - 1,
        CASE
            WHEN ST_DWithin(ST_EndPoint(input_geom.geom),
network_geom.geom, %s)
                THEN ST_ClosestPoint(network_geom.geom,
ST_EndPoint(input_geom.geom))
            ELSE ST_EndPoint(input_geom.geom)
        END
    )
    END AS geom,
    network_geom.geom AS network
FROM input_geom
CROSS JOIN network_geom
)
SELECT ST_AsText(
    ST_SnapToGrid(
        CASE
            WHEN endpoint_adjusted.network IS NULL THEN
endpoint_adjusted.geom
            ELSE ST_Snap(endpoint_adjusted.geom,
endpoint_adjusted.network, %s)
        END,
        %s
    )
)
FROM endpoint_adjusted;
"""
params = [geom_wkt, self.epsg, size, size, size, size]
else:
    query = f"""
        WITH input_geom AS (
            SELECT ST_GeomFromText(%s, %s) AS geom
        ),
        network_geom AS (
            SELECT ST_Collect(geom) AS geom
            FROM {table_name}
            WHERE id <> %s
        ),
        endpoint_adjusted AS (
            SELECT
                CASE
                    WHEN network_geom.geom IS NULL THEN input_geom.geom
                    ELSE ST_SetPoint(
                        ST_SetPoint(
                            input_geom.geom,
                            0,
                            CASE
                                WHEN ST_DWithin(ST_StartPoint(input_geom.geom),
network_geom.geom, %s)
                                    THEN ST_ClosestPoint(network_geom.geom,
ST_StartPoint(input_geom.geom))
                                ELSE ST_StartPoint(input_geom.geom)
                            END

```

```

        ),
        ST_NPoints(input_geom.geom) - 1,
        CASE
            WHEN ST_DWithin(ST_EndPoint(input_geom.geom),
network_geom.geom, %s)
            THEN ST_ClosestPoint(network_geom.geom,
ST_EndPoint(input_geom.geom))
            ELSE ST_EndPoint(input_geom.geom)
        END
    )
    END AS geom,
    network_geom.geom AS network
FROM input_geom
CROSS JOIN network_geom
)
SELECT ST_AsText(
    ST_SnapToGrid(
        CASE
            WHEN endpoint_adjusted.network IS NULL THEN
endpoint_adjusted.geom
            ELSE ST_Snap(endpoint_adjusted.geom,
endpoint_adjusted.network, %s)
        END,
        %s
    )
)
FROM endpoint_adjusted;
"""
    params = [geom_wkt, self.epsg, exclude_id, size, size, size, size]

    self.cur.execute(query, params)
    return self.cur.fetchone()[0]

def exists_by_id(self, table_name, row_id):
    query = f"SELECT EXISTS (SELECT 1 FROM {table_name} WHERE id = %s);"
    self.cur.execute(query, [row_id])
    return self.cur.fetchone()[0]

def has_interior_intersection(self, table_name, geom_wkt):
    query = f"""
        SELECT EXISTS (
            SELECT 1 FROM {table_name}
            WHERE ST_Relate(ST_GeomFromText(%s, %s), geom, 'T*****')
        );
    """
    self.cur.execute(query, [geom_wkt, self.epsg])
    return self.cur.fetchone()[0]

def get_interior_intersections(self, table_name, geom_wkt, exclude_id=None):
    query = f"""
        SELECT id
        FROM {table_name}
        WHERE ST_Relate(ST_GeomFromText(%s, %s), geom, 'T*****')
    """

```

```
"""
params = [geom_wkt, self.epsg]

if exclude_id is not None:
    query += " AND id <> %s"
    params.append(exclude_id)

query += " ORDER BY id"
self.cur.execute(query, params)
return [row[0] for row in self.cur.fetchall()]

def has_interior_intersection_excluding_id(self, table_name, geom_wkt, row_id):
    return bool(self.get_interior_intersections(table_name, geom_wkt,
exclude_id=row_id))
```